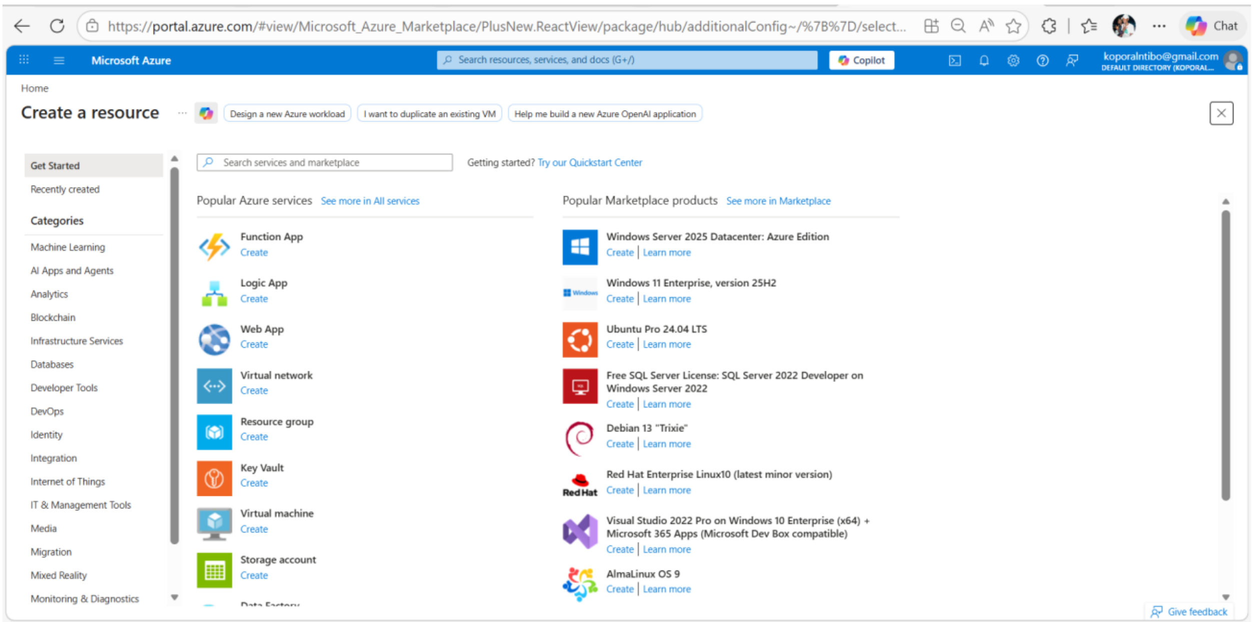
Chapter 2 Lab: Azure Compute Services

Task 1: Create a Linux Virtual Machine (IaaS)



Start by clicking **"Create a resource"** and use the search bar to find **"Virtual Machine"**, then select **"Create."** This will open the Virtual Machine setup page. From there, choose your subscription (for example, **Azure subscription 1**) and create a new resource group by selecting the **"Create new"** option and giving it an appropriate name.

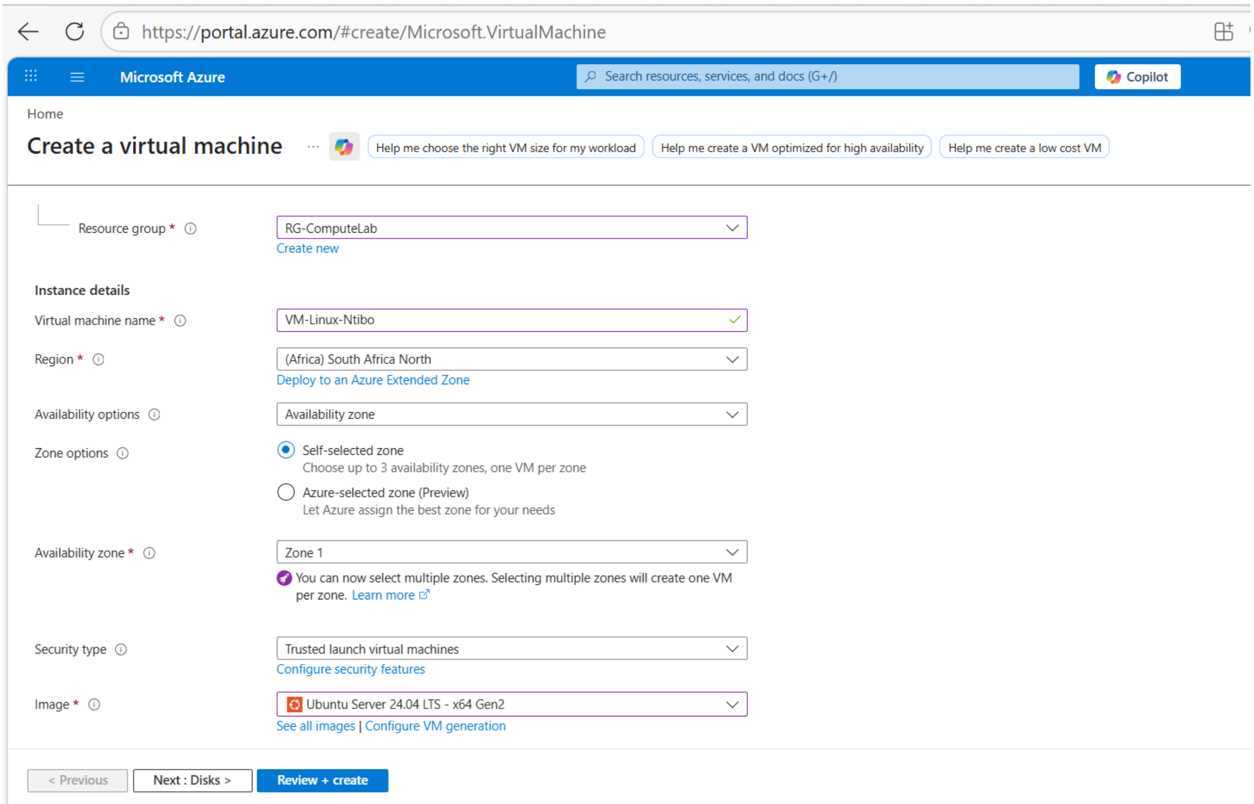
Next, assign a name to your Virtual Machine, usually based on your surname as instructed. After that, select the region, such as **South Africa North**, to ensure better performance based on your location.

For the image, you can choose **Ubuntu Server 22.04 LTS** as required, but if it is not available, **Ubuntu Server 24.04 LTS** can be used as an alternative. This version provides a secure and stable Linux environment with long-term support and is well optimized for Azure, offering improved performance and faster startup times.

When selecting the size, choose a cost-effective option like **B2ats_v2**, which is suitable for low-demand workloads such as small applications, development environments, or basic web servers.

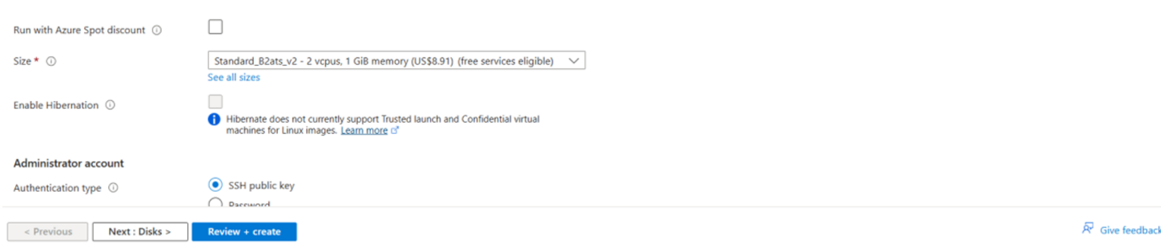
Then, configure the administrator account by setting the username as **"azureuser"** and creating a strong password to ensure secure access to the virtual machine.

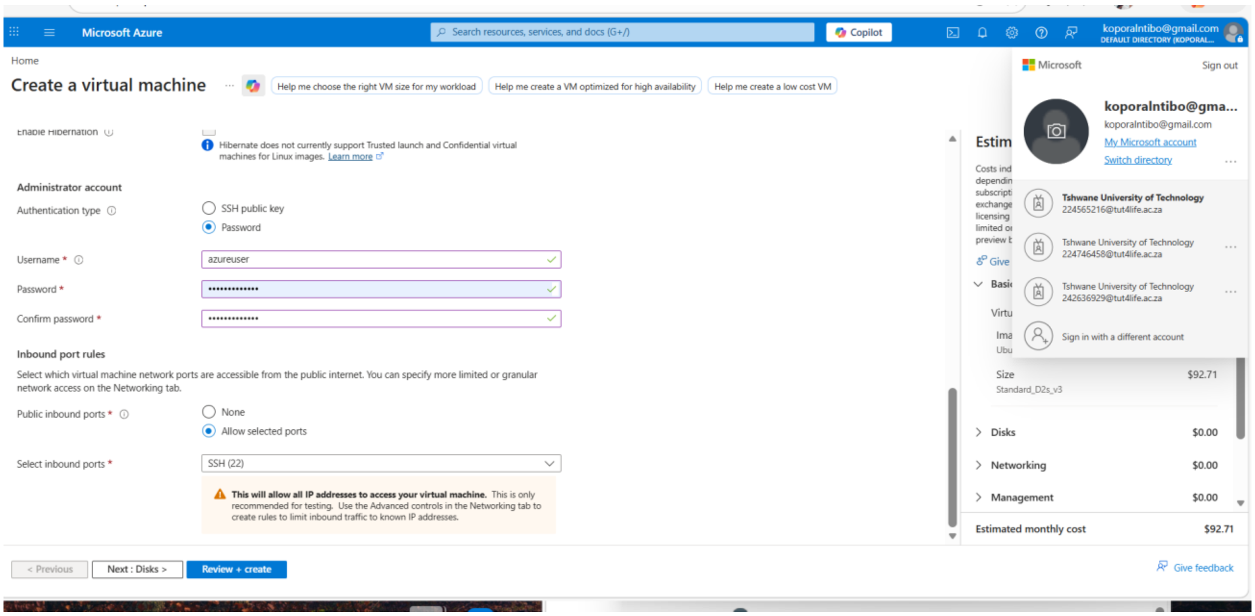
Finally, configure the networking settings to allow **SSH (port 22)** so that you can connect remotely to the Linux machine. Once everything is set, proceed to create the Virtual Machine.



• I used the default **Azure subscription 1** because my account did not have access to the Azure for Students option. This subscription still provided the necessary features to complete the lab tasks.

• I selected the region **South Africa North** since it is closest to my location, making it suitable as a data center for better performance.



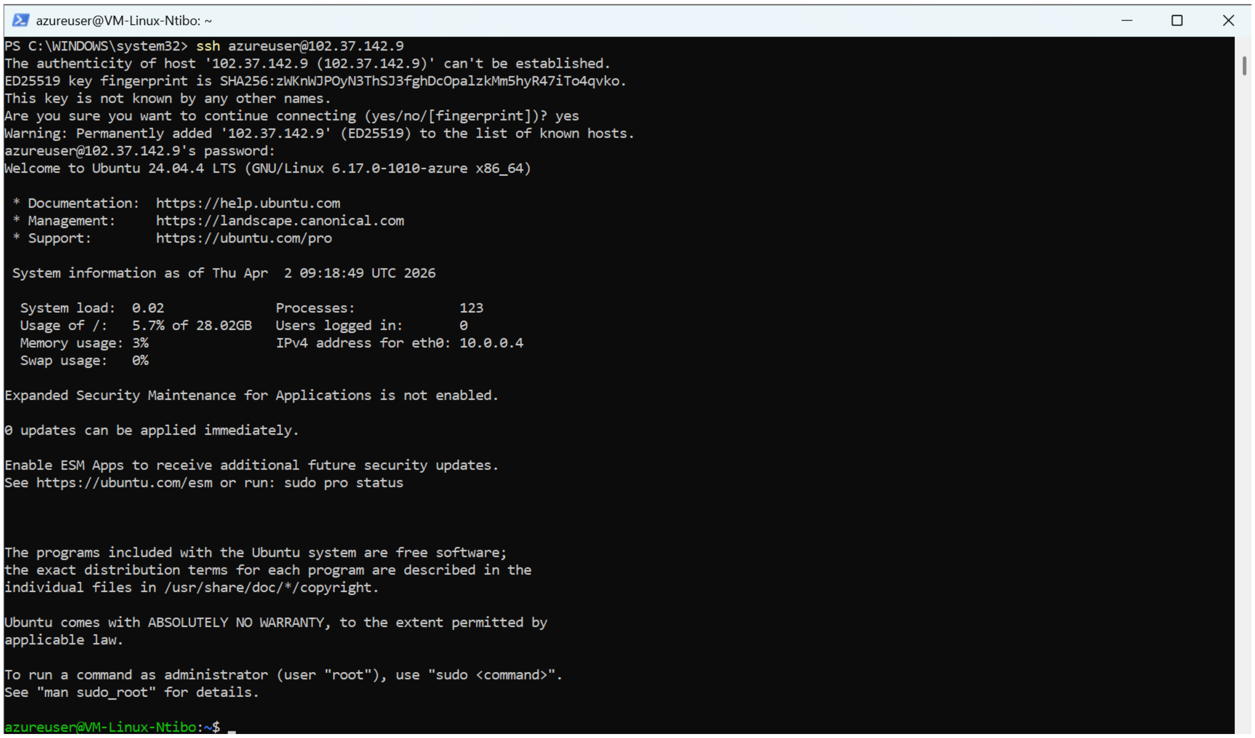


After successfully creating and deploying the Virtual Machine, you can use **Windows PowerShell** to manage and access it. PowerShell is useful for automation, efficiency, and seamless integration with Microsoft services.

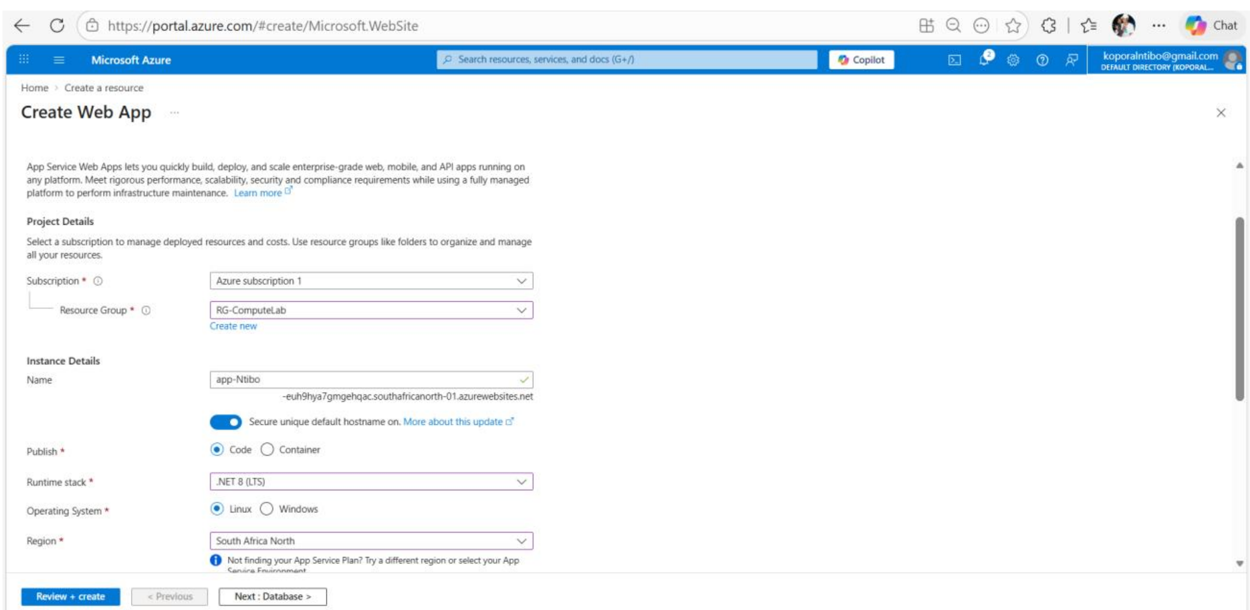
To connect to the Virtual Machine, open PowerShell and enter the command:
ssh azureuser@PublicIP

In this command, **azureuser** represents the administrator username you created, while **PublicIP** refers to the virtual machine's public IP address, which can be found once the VM is running.

You will then be prompted to enter the administrator password, after which you will gain access to the Virtual Machine's environment.



Task 2: Deploy Azure App Service (PaaS)



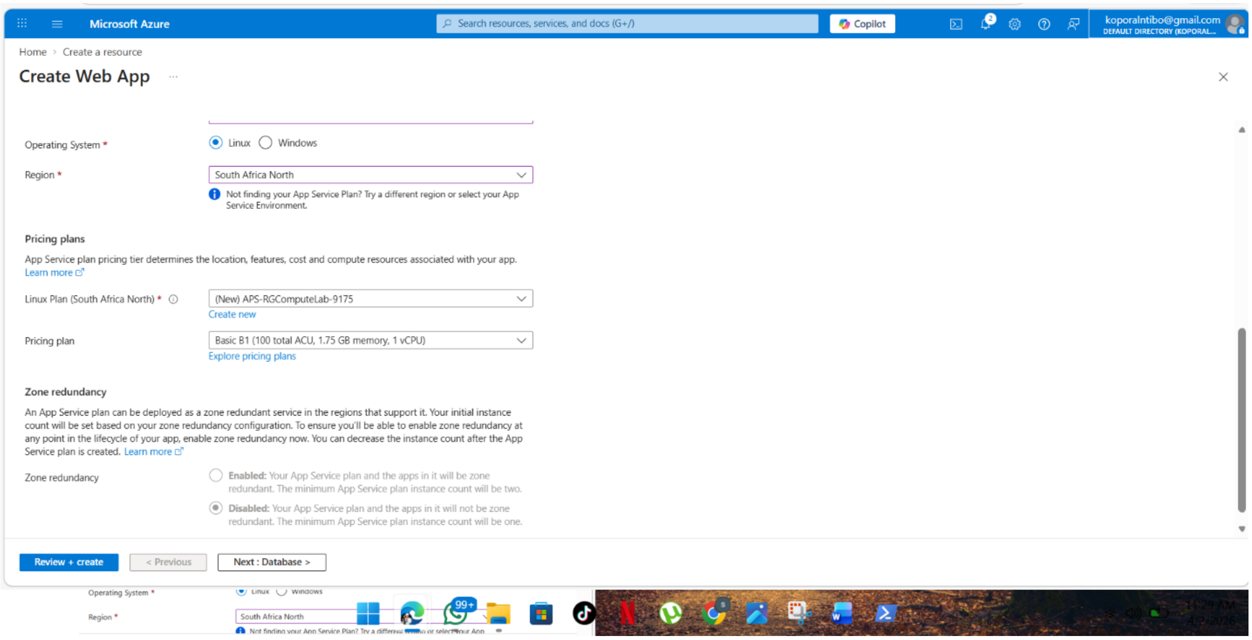
You will be required to deploy a web application without managing servers, using Platform as a Service (PaaS). You can do this by creating a new resource, and searching for App Service in the search bar, then clicking create.

You will be then required to fill in the required details to create the Web App. This includes Project details, where subscriptions and resource groups are needed. Selecting your subscription "Azure for Students" and Resource Group, the one created on the previous task. Instance details will also be required, where naming your Web App using your surname and selecting "Code" as publish option. Code – the standard choice for deploying applications built with specific programming languages and frameworks (such as .NET and Node.js) directly to a fully managed hosting environment. Runtime stack = .NET (LTS), as it ensures maximum stability, security, and performance for production applications.

In my case using .NET 8 (LTS) which provides long-term stability and support, significant performance enhancements, and built-in features optimized for cloud native development, instead of the recommended .NET 6 or Node.js.

Feature / Criteria	**.NET 8**	**.NET 6**	**Node.js**
Performance	Highest throughput, improved JIT, native AOT compilation	Stable, but slower than .NET 8	Fast for I/O-bound tasks, weaker for CPU-heavy
Azure Integration	Deep integration with Azure Functions, App Service, AKS, Cosmos DB	Supported, but lacks latest optimizations	Supported, especially with serverless (Functions)
Microservices	Strong with ASP.NET Core, gRPC, Dapr, Kubernetes	Good, but less optimized for container workloads	Excellent for lightweight, event-driven microservices
Scalability	Enterprise-grade, handles millions of requests with low latency	Solid, but less efficient under extreme load	Scales horizontally well, but requires more tuning
Developer Experience	Modern C# features, better tooling in VS Code & Visual Studio	Mature ecosystem, but fewer new features	Huge npm ecosystem, fast development

Feature / Criteria	.NET 8	.NET 6	Node.js
Best Use Cases	Enterprise APIs, financial systems, cloud-native apps	Legacy modernization, stable enterprise apps	Real-time apps, APIs, streaming services, lightweight backends

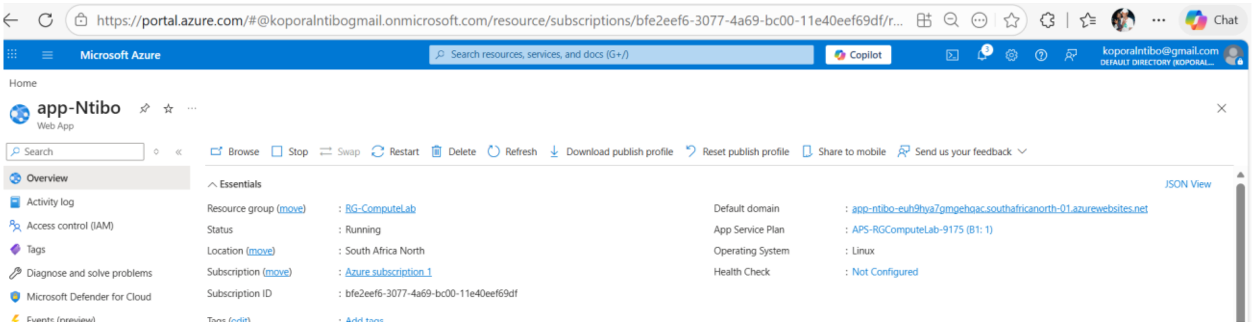


App Service Plan Selection

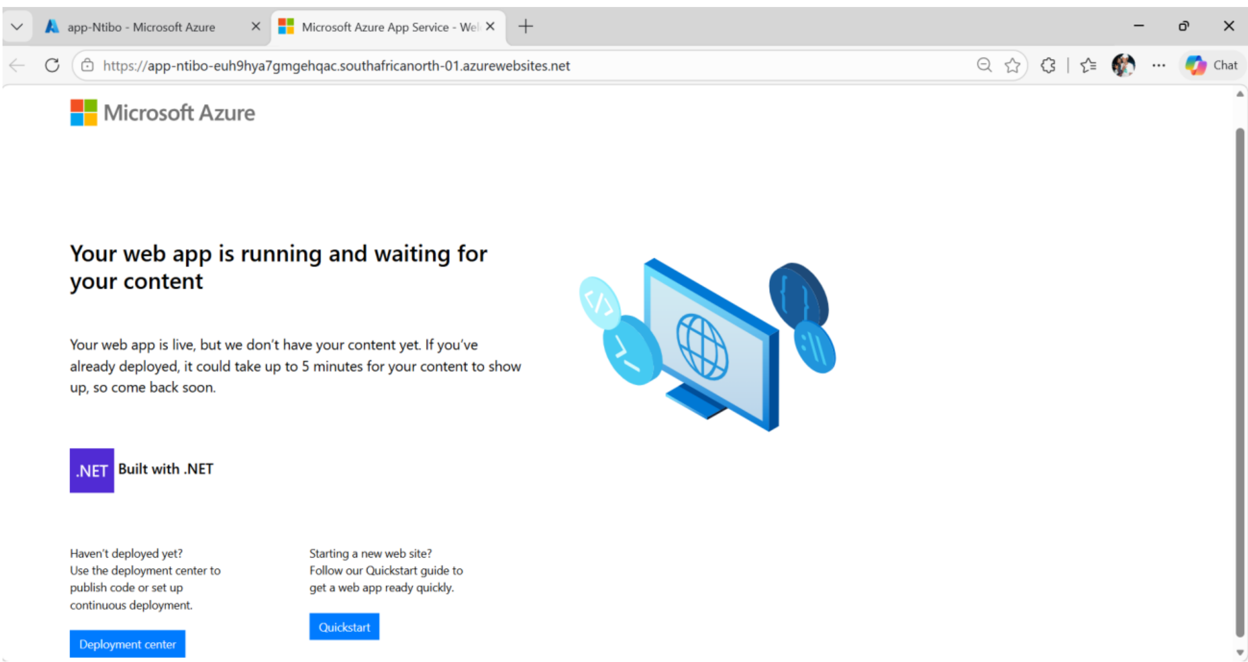
App Service plan pricing determines the location, features, cost, and compute resources associated with an application. In this project, the **Basic B1 plan** was selected as it is a cost-effective, dedicated compute tier suitable for low-traffic applications, as well as development and testing environments.

The same pricing plan was used for deploying the application because the **Free (F1) tier**, which was initially recommended, was not available in the Azure account.

After configuring the App Service plan, the Web App was successfully created and deployed.



Using the Web App’s default domain after deployment will allow you to see a default web page of your web app.



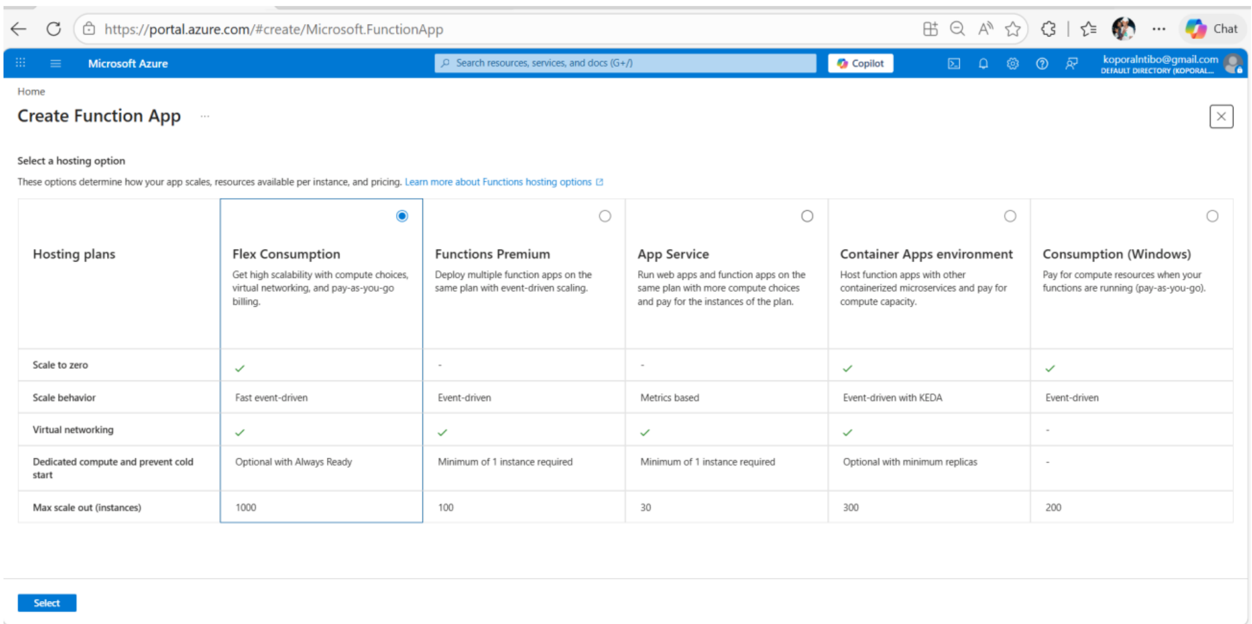
Task 3: Create Azure Function (Serverless)

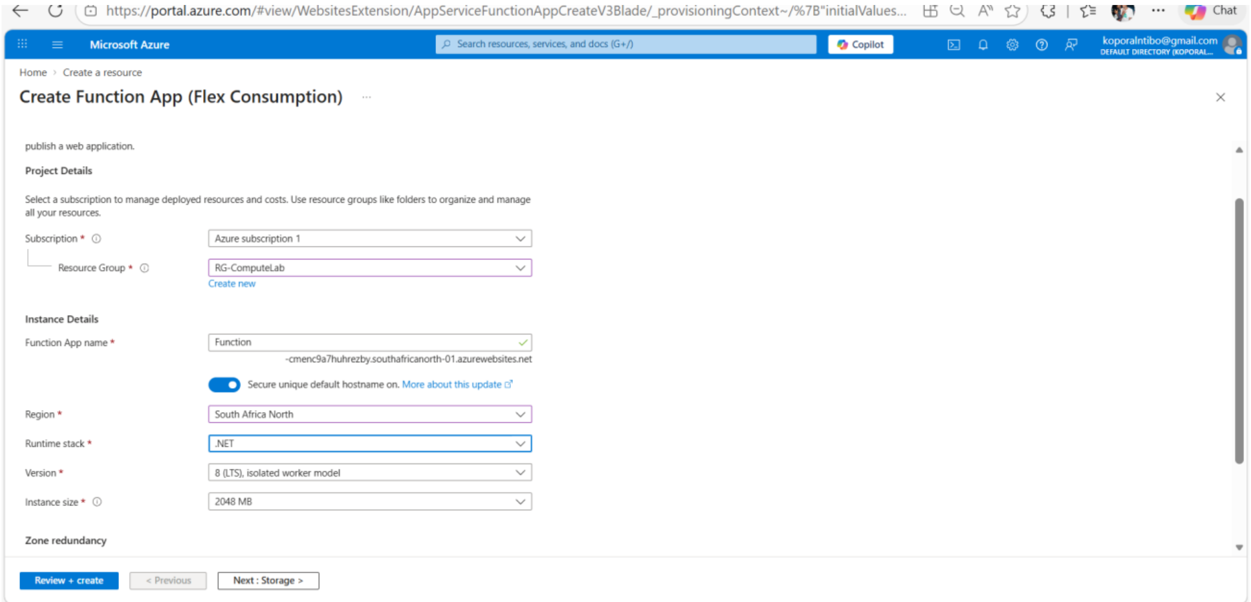
Serverless Computing and Azure Functions

Serverless computing is an execution model in which developers deploy code as functions, while the cloud provider automatically manages the underlying infrastructure. Although the term “serverless” is used, servers are still involved; however, they are abstracted away from the developer, allowing a greater focus on application logic rather than system management.

Azure Functions, a service provided by Microsoft, is a serverless compute solution that enables the execution of event-driven code without the need to manage servers. It automatically scales based on demand, ensuring efficient resource utilization. Additionally, it integrates seamlessly with other Azure services and follows a pay-per-execution pricing model, meaning users are only charged for the time their code runs.

This makes Azure Functions particularly suitable for building APIs, automation tasks, and microservices.

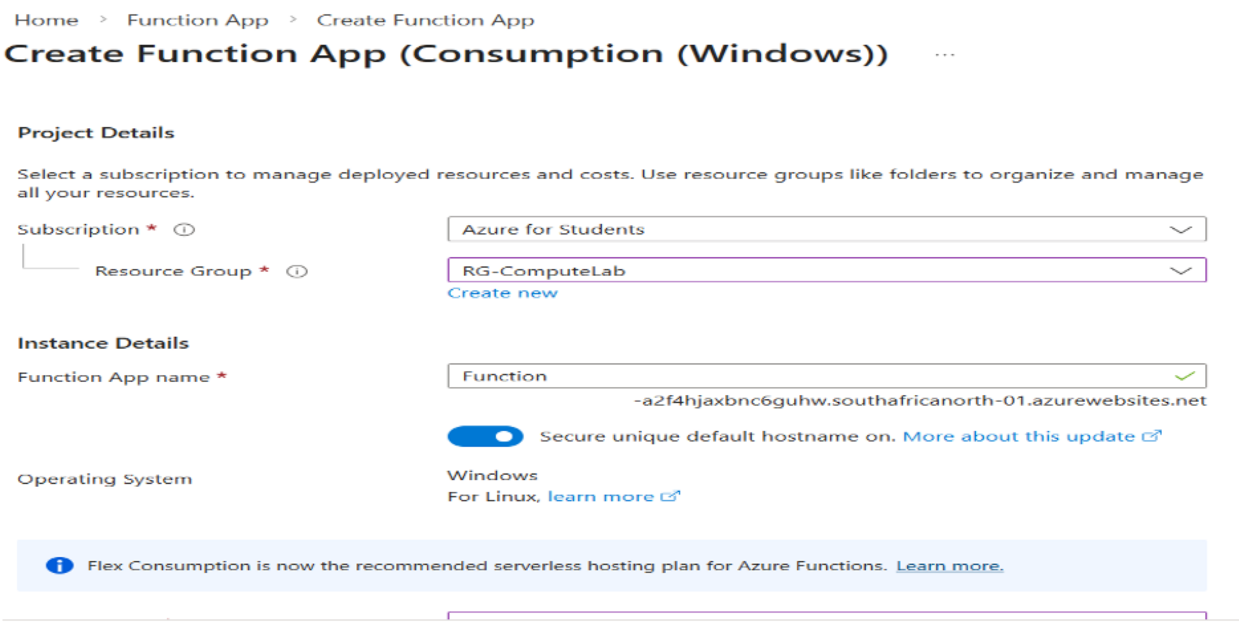




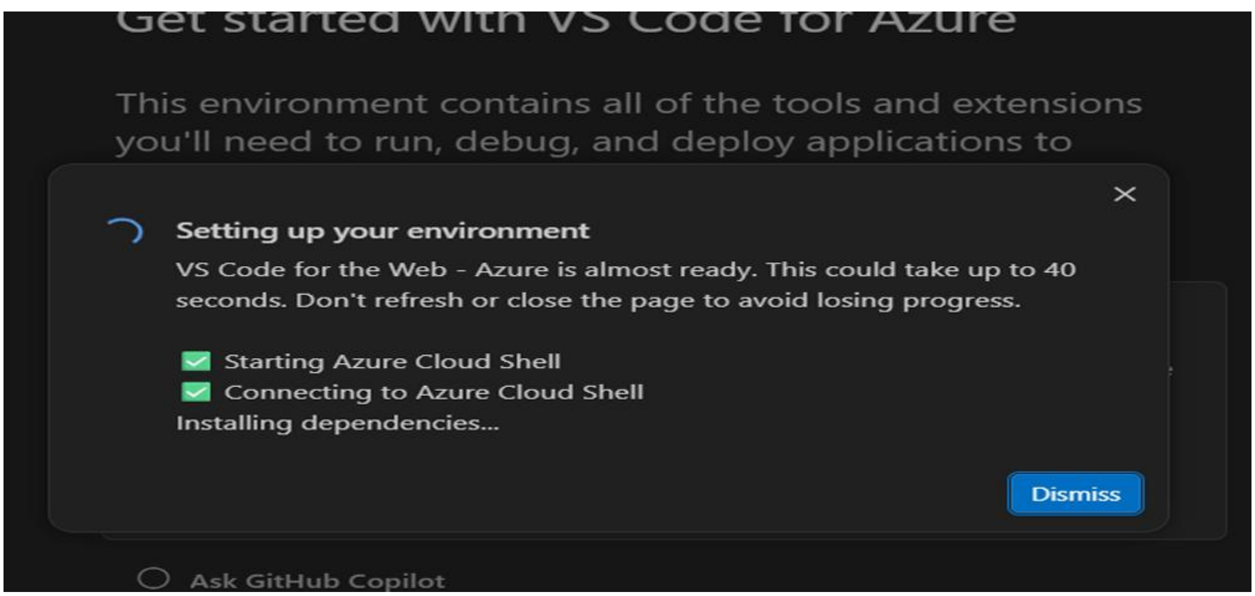
NET 8 Configuration and Instance Size

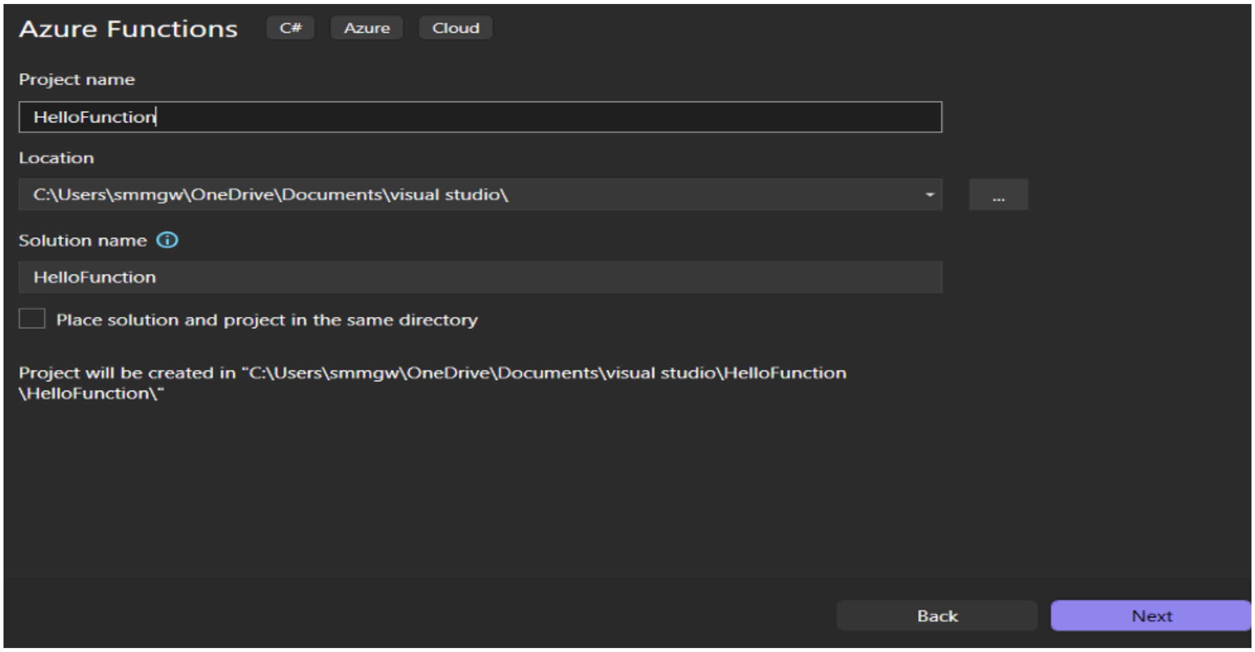
.NET 8 (LTS) provides long-term stability and support, along with significant performance improvements and built-in features that are optimized for cloud-native development. These enhancements make it a suitable choice for building scalable and efficient applications in modern cloud environments.

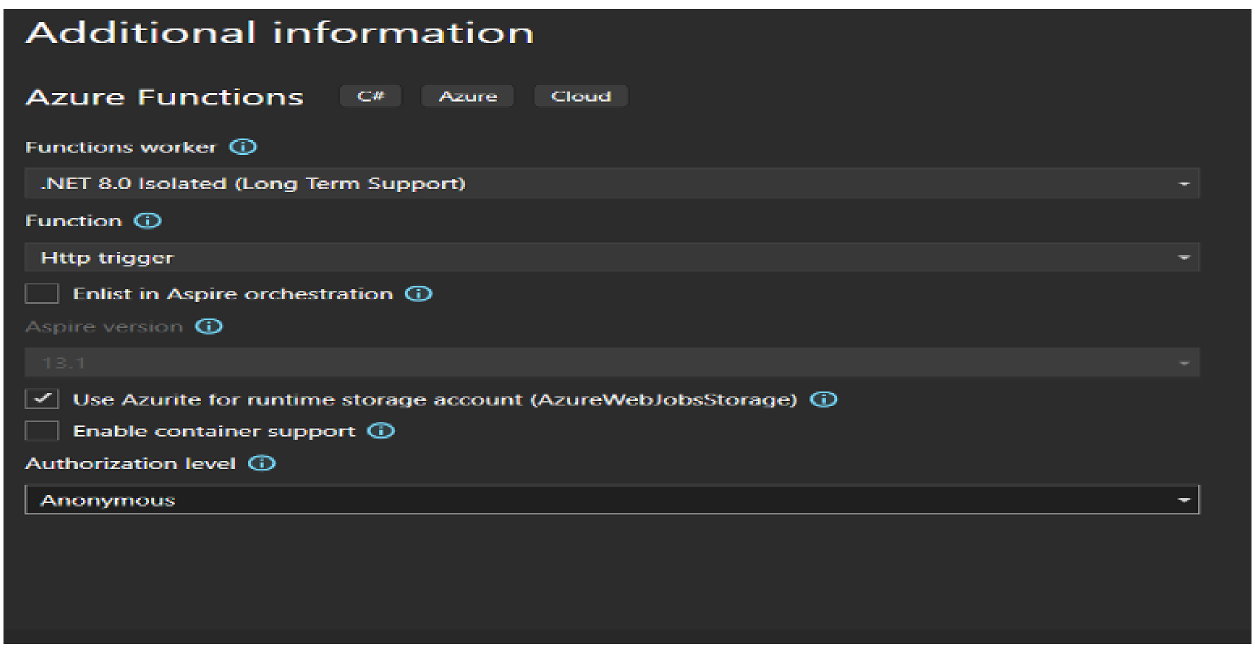
An instance size of **2048 MB** was selected to support the runtime stack version used during the creation of the function. This ensures adequate memory allocation for smooth execution and improved performance of the application.



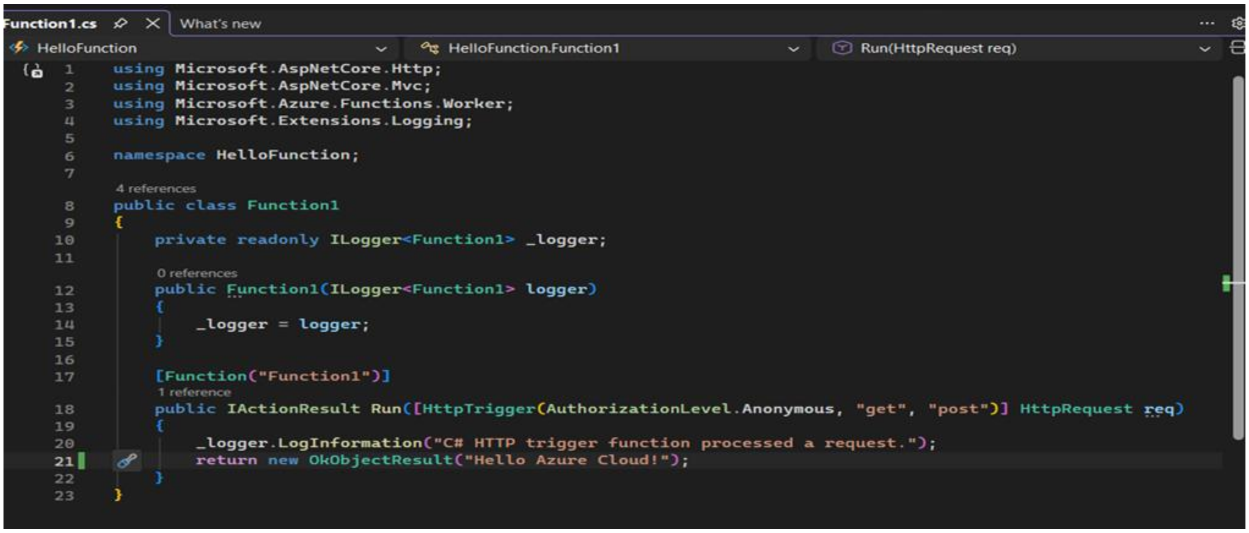
To create a function for Function App I used Visual Studio Code, because it provides a lightweight, cross-platform environment with the Azure Functions extension, enabling local development, debugging, and seamless deployment to Azure. It supports multiple languages (C#, JavaScript, Python, etc.) and integrates tightly with Azure services, making it ideal for modern serverless workflows.







Creating Function in Visual Studio

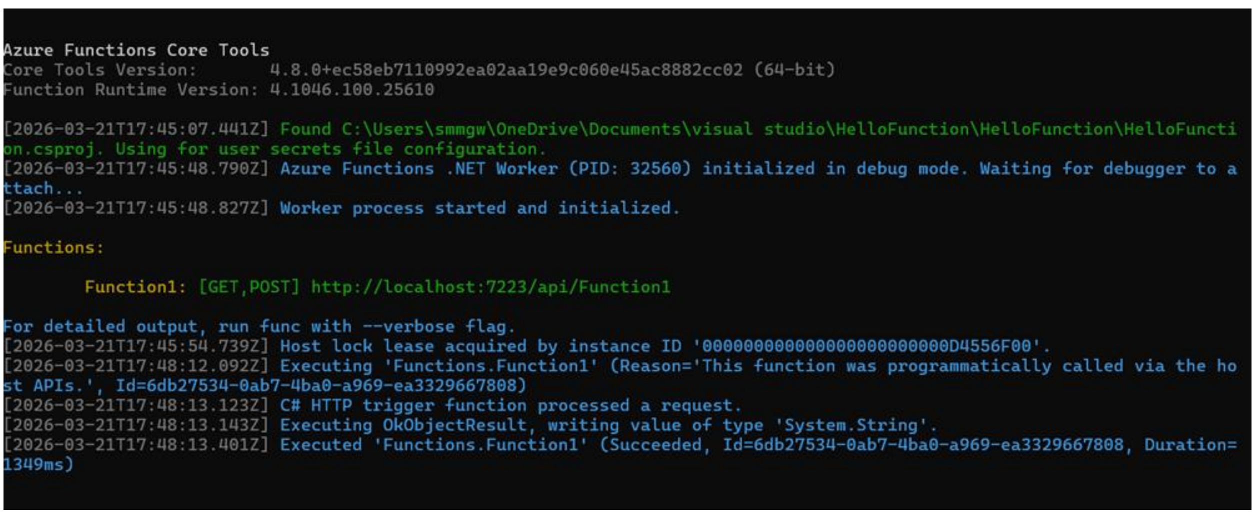



```csharp
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;

namespace HelloFunction;

4 references
public class Function1
{
    private readonly ILogger<Function1> _logger;

    0 references
    public Function1(ILogger<Function1> logger)
    {
        _logger = logger;
    }

    [Function("Function1")]
    1 reference
    public IActionResult Run([HttpTrigger(AuthorizationLevel.Anonymous, "get", "post")] HttpRequest req)
    {
        _logger.LogInformation("C# HTTP trigger function processed a request.");
        return new OkObjectResult("Hello Azure Cloud!");
    }
}
```

Adding Code Inside the Function Editor

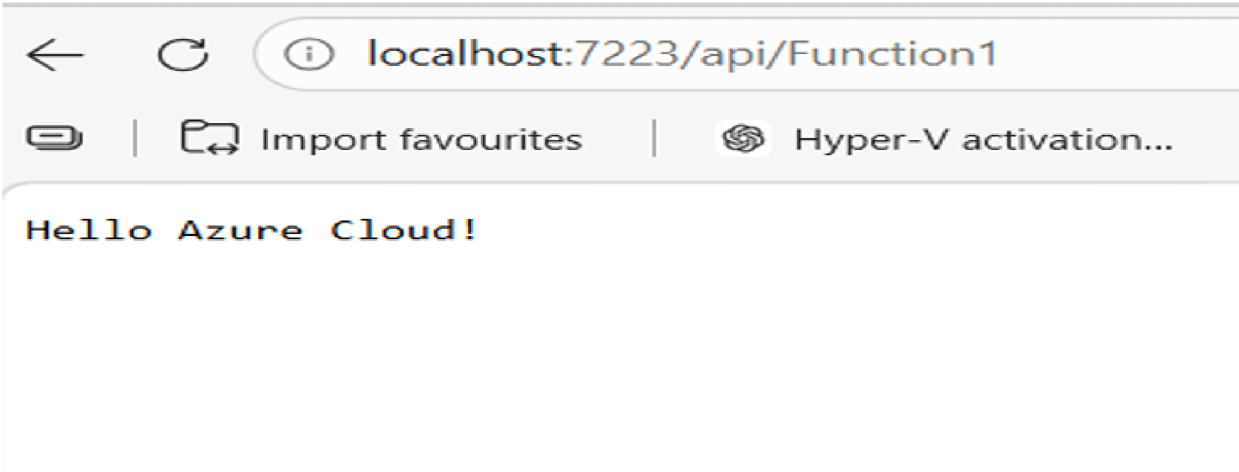
After creating the function, code was added inside the function editor to define the logic of the application. The function was configured to return a simple HTTP response message, confirming that the function is running successfully.

This step is important as it allows the developer to test and verify that the Azure Function is correctly deployed and functioning as expected.



```
Azure Functions Core Tools
Core Tools Version:       4.8.0+ec58eb7110992ea02aa19e9c060e45ac8882cc02 (64-bit)
Function Runtime Version: 4.1046.100.25610

[2026-03-21T17:45:07.441Z] Found C:\Users\smmgw\OneDrive\Documents\visual studio\HelloFunction\HelloFunction\HelloFuncti
on.csproj. Using for user secrets file configuration.
[2026-03-21T17:45:48.790Z] Azure Functions .NET Worker (PID: 32560) initialized in debug mode. Waiting for debugger to a
ttach...
[2026-03-21T17:45:48.827Z] Worker process started and initialized.

Functions:

        Function1: [GET,POST] http://localhost:7223/api/Function1

For detailed output, run func with --verbose flag.
[2026-03-21T17:45:54.739Z] Host lock lease acquired by instance ID '0000000000000000000000000D4556F00'.
[2026-03-21T17:48:12.092Z] Executing 'Functions.Function1' (Reason='This function was programmatically called via the ho
st APIs.', Id=6db27534-0ab7-4ba0-a969-ea3329667808)
[2026-03-21T17:48:13.123Z] C# HTTP trigger function processed a request.
[2026-03-21T17:48:13.143Z] Executing OkObjectResult, writing value of type 'System.String'.
[2026-03-21T17:48:13.401Z] Executed 'Functions.Function1' (Succeeded, Id=6db27534-0ab7-4ba0-a969-ea3329667808, Duration=
1349ms)
```

Running code in Visual Studio

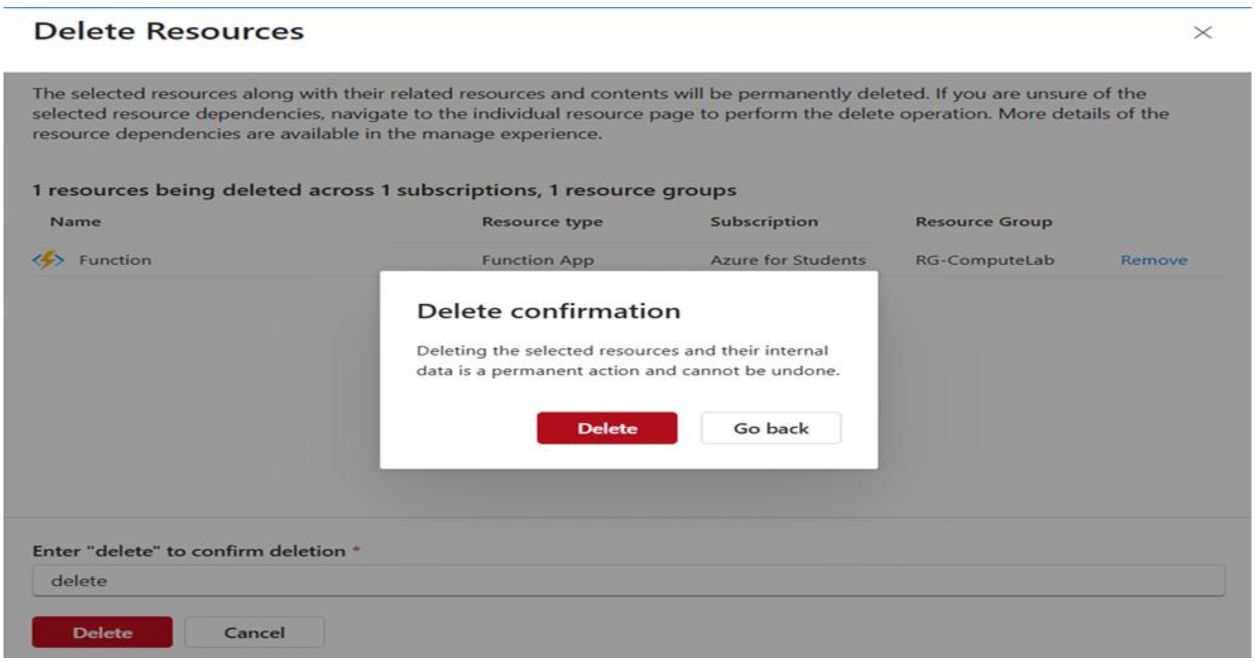


Testing the Function in a browser (any browser) Azure Function run only when triggered, and they auto-scale automatically.

Comparison: VM (IaaS) vs App Service (PaaS) vs Function (Serverless)

Feature	VM (IaaS)	App Service (PaaS)	Function (Serverless)
OS Control	Yes	No	No
Scaling	Manual / Auto	Auto	Auto
Billing	Per minute	Per hour	Per execution
Best For	Full control apps	Web apps	Event-driven tasks

Task 4: Cleanup



Resource Cleanup

Cloud resources incur costs; therefore, deleting them when they are no longer needed helps an organization reduce expenses and optimize the use of cloud storage and computing resources.